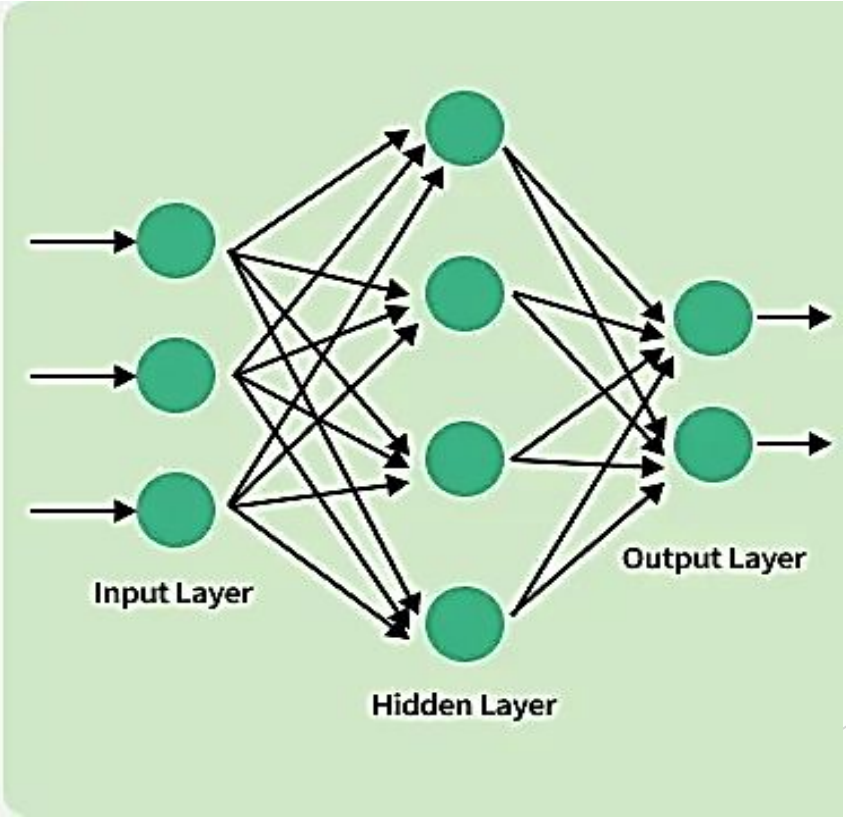
Deep Feedforward Neural Networks (FNNs)

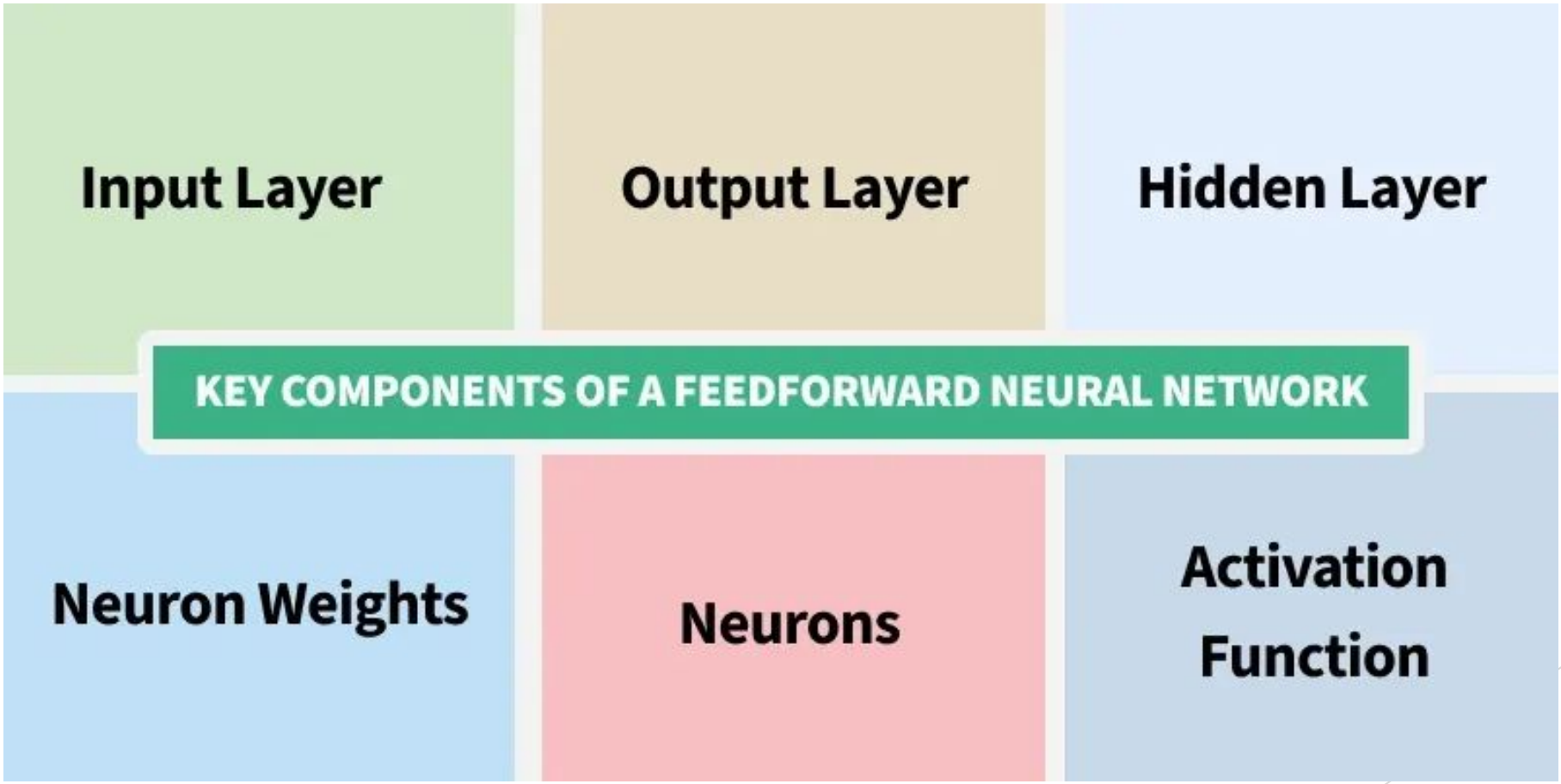
Feedforward Neural Networks (FNNs)

What Is A Feedforward Neural Network?

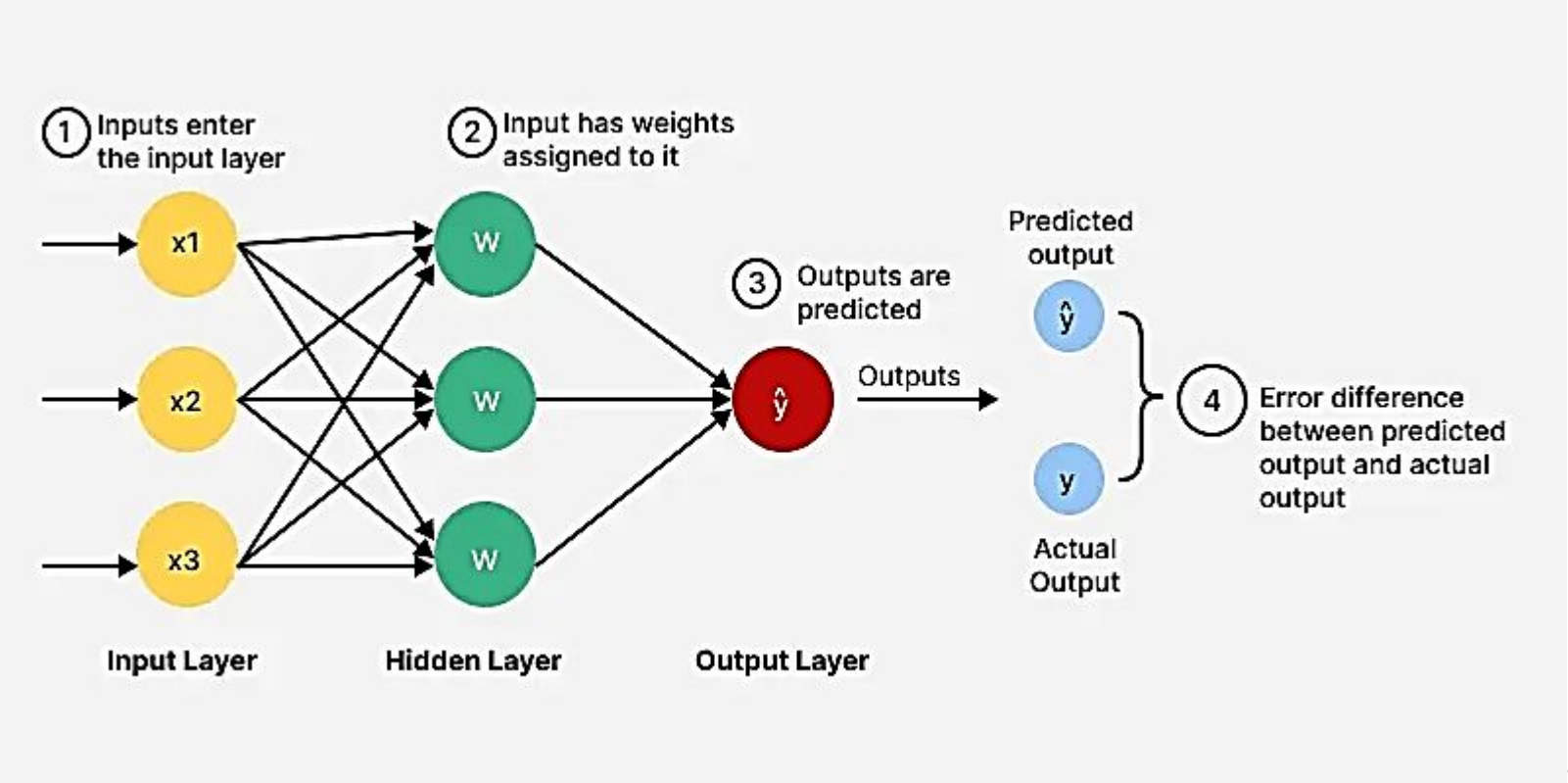
1. A neural network where data flows one way i.e. from input to output.
2. No loops or feedback.
3. Used for tasks like pattern recognition (e.g image and speech).



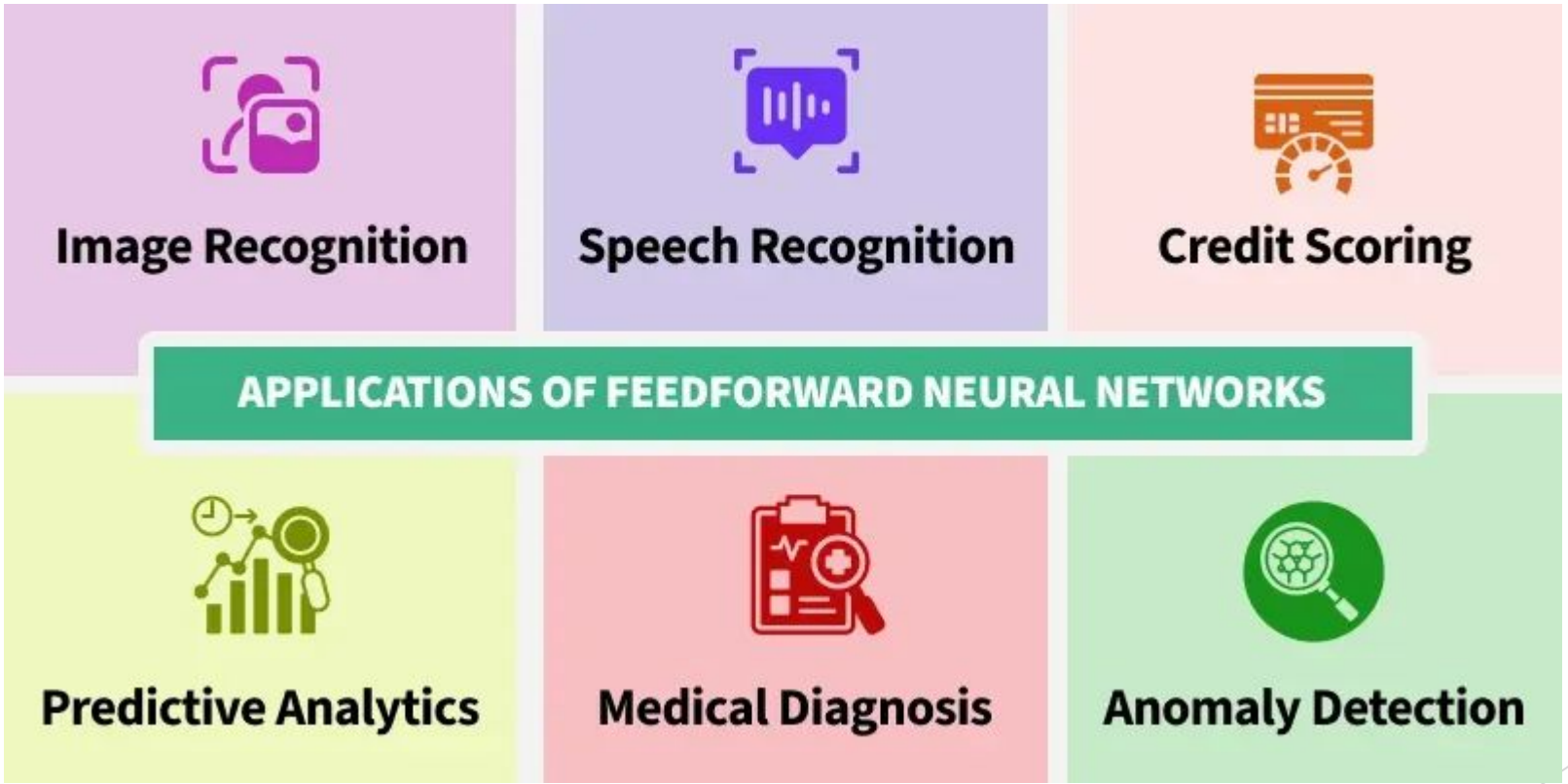
Feedforward Neural Networks (FNNs)



Feedforward Neural Networks (FNNs)

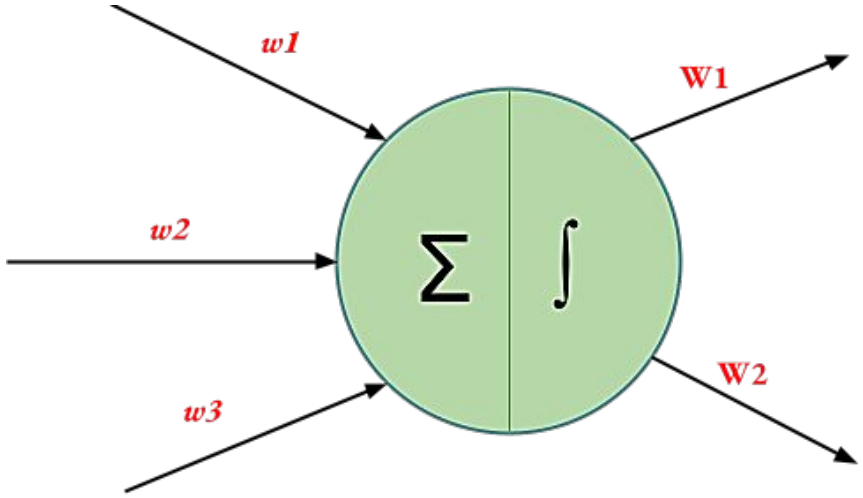


Feedforward Neural Networks (FNNs)



Weight Initialization Techniques

- It is crucial to initialize the weights appropriately to ensure a model with high accuracy.
- Vanishing Gradient problem or the Exploding Gradient problem might rise if weights are not properly initialized.
- fan_{in} = Number of input paths towards the neuron
- fan_{out} = Number of output paths towards the neuron

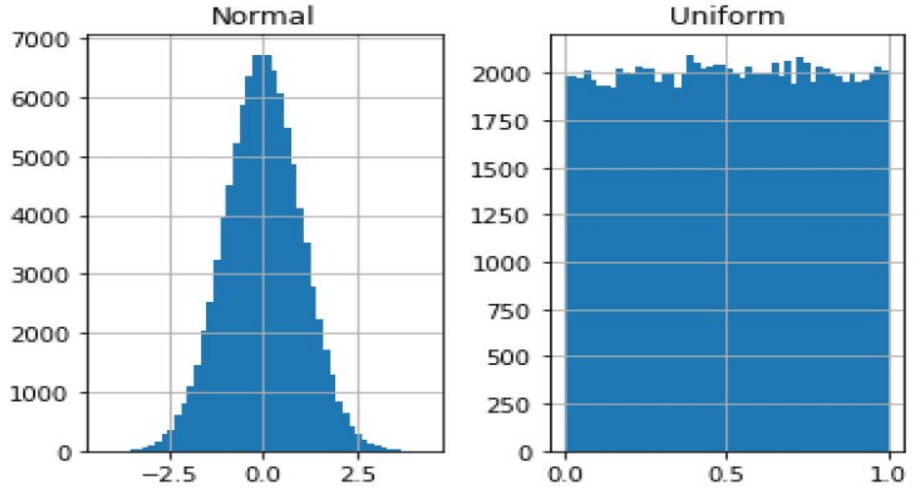


Zero Initialization

- All the weights are assigned zero as the initial value.
- Why fails?
  - Highly ineffective as neurons learn the same feature during each iteration (symmetry problem).
  - Constant initializations are not preferred.
  - All neurons in a given layer produce the same output and receive the same gradient during backpropagation, making them functionally identical.
  - Zero Gradients (depending on activation): If all weights are zero, for many common activation functions (like ReLU), all activations will also be zero.
- When to use?
  - Initializing **biases** to zero does not cause the symmetry problem, as the randomness in the weight initialization is enough to break the symmetry across neurons.

Random Initialization

- Assigns random values except for zeros as weights to neuron paths.
- Overfitting, Vanishing Gradient Problems might rise due to randomly assigned weights.
- **Random Initialization can be of two kinds:**
  - Random Normal
    - clusters data around a central mean
  - Random Uniform
    - all outcomes within a range are equally likely

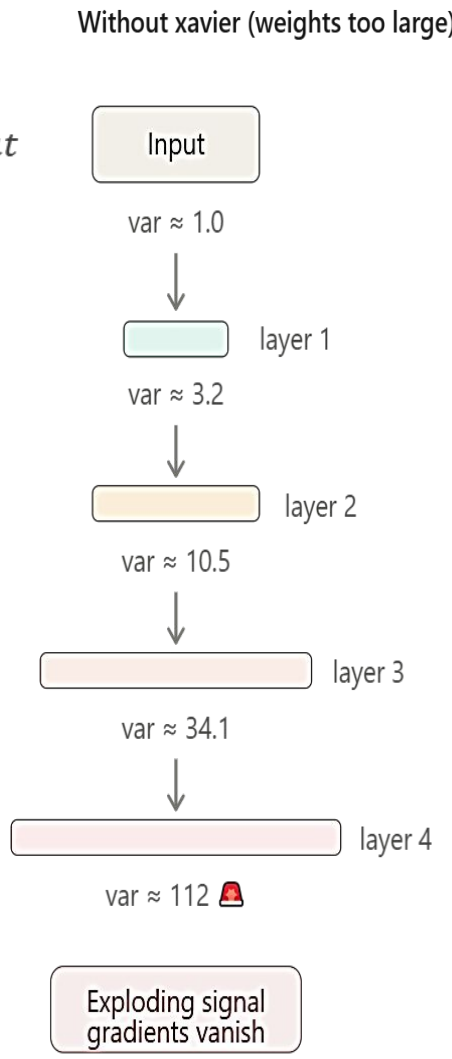


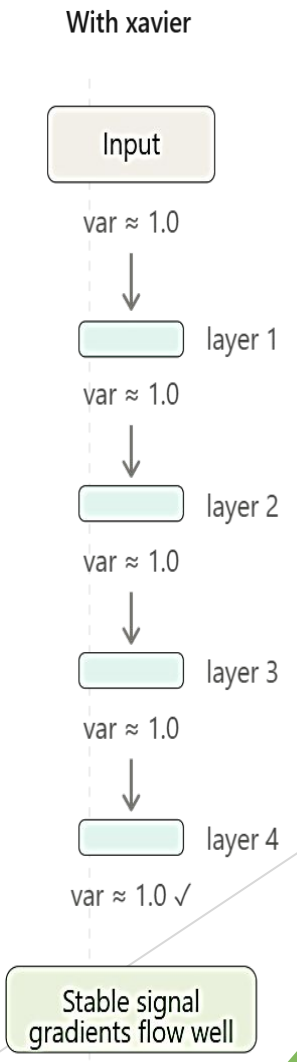
Xavier/Glorot Initialization

- Problem:
  - When you train a neural network, you start by randomly initializing weights. If those weights are too small, signals shrink as they pass through layers (vanishing gradients). If too large, signals explode (exploding gradients). Either way, your network won't learn properly.
- Xavier initialization picks initial weights that are just right; keeping signal variance roughly the same across layers.
- The intuition:
  - Imagine you're passing a ball through a series of rooms. Each room either amplifies or shrinks the ball. If the room makes it too big, it bursts through the ceiling. Too small, it disappears. Xavier's idea: design each room to pass the ball through at roughly the same size.

Xavier/Glorot Initialization

- For a layer with n_{in} inputs and n_{out} outputs:
  - Uniform version:
    $W \sim Uniform(-limit, +limit)$
    $$limit = sqrt\left(\frac{6}{(n_{in} + n_{out})}\right)$$
  - Normal version:
    $W \sim Normal(0, std)$
    $$std = sqrt(\frac{2}{(n_{in} + n_{out})})$$
- Both are equivalent; PyTorch uses the uniform version by default.
- Choose weights such that the variance of outputs = variance of inputs for every layer.





Xavier/Glorot Initialization

- For a layer with n_{in} inputs and n_{out} outputs:
  - Uniform version:

$$W \sim Uniform(-limit, +limit)$$

$$limit = sqrt\left(\frac{6}{(n_{in} + n_{out})}\right)$$

  - Normal version:

$$W \sim Normal(0, std)$$

$$std = sqrt(\frac{2}{(n_{in} + n_{out})})$$

- Both are equivalent; PyTorch uses the uniform version by default.

He Initialization

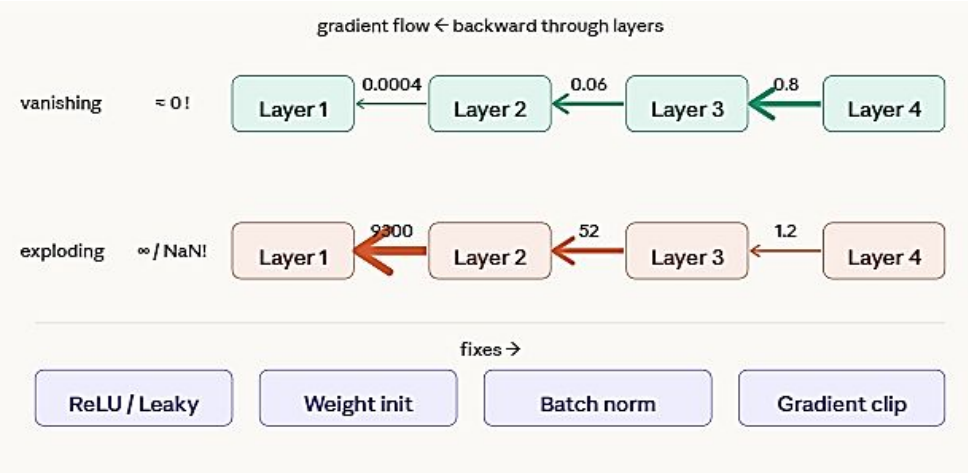
- Xavier assumes activations are symmetric around zero (like tanh, sigmoid). But ReLU is different:
  - ReLU **kills all negative values**; setting them to zero. This means roughly **half the neurons are dead** at any moment. Xavier doesn't account for this, so the variance it targets ends up being too small, and signals still shrink across deep ReLU networks.
- He initialization fixes this by saying: *"If half the neurons are off, double the variance to compensate."*
- *Xavier's starting constraint was:* $Var(W) = \frac{2}{n\_in + n\_out}$ average of forward + backward.
- *He (Kaiming) made a simpler, stronger choice "only care about the forward pass", and correct for ReLU's dead neurons.*
- *Forward pass constraint (same as Xavier):* $Var(W) = \frac{1}{n_{in}}$
- *ReLU correction — half neurons are zeroed, so double it:* $Var(W) = \frac{2}{n_{in}}$
- *No backward pass averaging.*

When to use what

Activation	Recommended Init
Sigmoid, Tanh	Xavier (Glorot)
ReLU	He (Kaiming). Xavier underestimates for ReLU
Linear / no activation	Xavier is fine

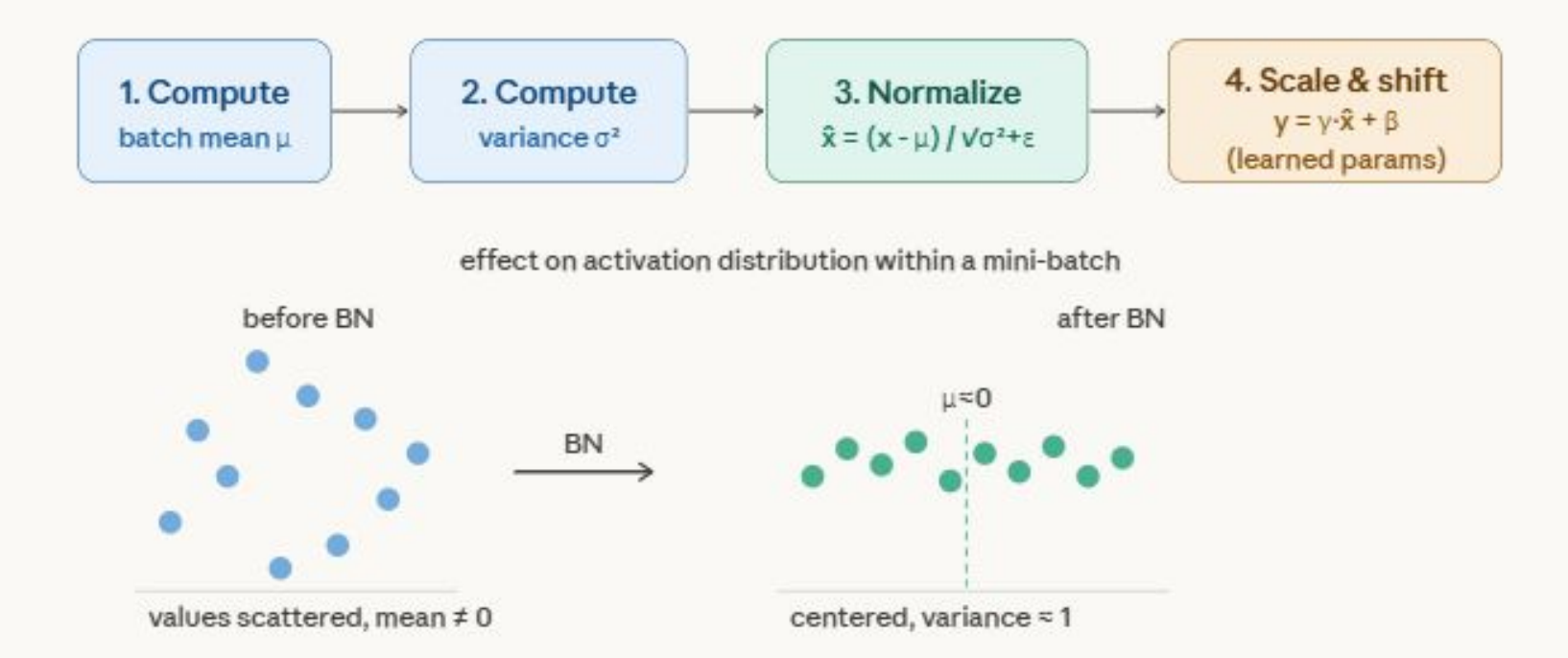
Vanishing / Exploding Gradients

- When you train a deep network, you calculate gradients and send them **backwards** through the layers (backpropagation).
- The problem: every layer **multiplies** the gradient by the weights and the derivative of the activation function.
  - If those values are consistently **< 1** → gradients shrink exponentially → **vanishing gradients** (early layers learn nothing)
  - If those values are consistently **> 1** → gradients grow exponentially → **exploding gradients** (weights blow up to infinity/NaN)
  - Think of it like passing a whisper through 100 people, either it fades to silence, or each person shouts louder and the last person goes deaf.



Batch Normalization

- Before feeding activations to the next layer, normalize them to have $\boldsymbol{mean} \approx \mathbf{0}$ and $\boldsymbol{variance} \approx \mathbf{1}$ within each mini-batch.
- This directly solves the "inputs to each layer keep shifting" problem (**internal covariate shift**).



Batch Normalization

- After normalizing, BN adds two learnable parameters per feature: γ (scale) and β (shift)
- This lets the network *undo* the normalization if needed, it doesn't force activations to always be zero-mean; it just gives the optimizer a stable starting point and lets it learn from there
- Batch normalization directly addresses vanishing/exploding gradients:
  - By keeping activations in a controlled range, the gradients flowing back through layers stay **well-scaled** — they don't shrink to zero or blow up
  - It also lets you use higher learning rates safely, which speeds up training significantly
  - Common practice: apply BN after the linear/conv layer but before the activation function
  - think of BN as a "reset button" between layers, it keeps every layer from receiving inputs that are too wild, so learning stays stable throughout the network.